N-Body Simulation of the Solar System: Testing Accuracy and Energy Conservation in Numerical Methods

Ishir Vikasith Nama[1]

[1]**s2862521@ed.ac.uk**

ABSTRACT

The N-body problem is a classical problem in physics that involves predicting the motion of multiple interacting bodies under gravity. In this project, I have made an OOP-style N-body simulation of a subset of the solar system (Sun, Mercury, Venus, Earth, Mars, Jupiter) using python. The underlying principle behind the simulation is Newton's universal law of gravitation. The program uses different integration methods (Beeman, Euler-Cromer and Direct Euler) simulate the positions of the bodies and predict when they align. The results show that Beeman's method is the best at predicting orbital periods and shows energy conservation the best out of all other integration methods. The results also show that a shortened timestep means better energy conservation.

Keywords: Period, conserved, alignment, vector, integration, body

INTRODUCTION

The motion of planets within the Solar System is governed by Newton's law of universal gravitation. While predicting motion of two bodies (i and j) can be solved analytically, it is much harder for 3 or more bodies. This is because the system behaves chaotically(1), meaning even a slight variance in initial conditions can cause the planets to move vastly differently over time. Numerical approaches are ideal for these systems and extensively used to study the motion of bodies in space.

My goal was to develop an N-body simulation of a subset of the Solar System (Sun, Mercury, Venus, Earth, Mars, Jupiter) using three different integration methods: Beeman, Euler-Cromer and Direct Euler. Each of these methods integrate to different degrees of accuracy and some conserve energy better than others. For the simulation, I made an Object-Oriented Program(OOP) in Python(2) which uses the N-body force formula to simulate the positions of the planets orbiting the Sun and animate them.

I made three experiments to test my simulation results for accuracy and energy conservation. In Experiment 1, I wrote a program that computed orbital periods for each planet using my simulation based on a given timestep. Then compared them to NASA's estimations(3) and calculated the percentage error. This is vital to understand how timestep choices influence accuracy. In Experiment 2, I wrote a program that tracked the total energy of the system over 25 simulated years and plots a graph of Total Energy(ΣE) vs time(t) and the RMS energy deviation(δE_{RMS}) vs time(t) depending on what integration method was chosen. Here, the RMS energy deviation is the metric that shows how well each integration method conserved energy. Finally, in Experiment 4, I extended the simulation to detect planetary alignments. I made a method which takes in a critical angle ϕ and runs the simulation. If all the planets make an angle $< \phi$ with a common line, then the time at that step is printed.

This report is structured as follows: the Methods section describes the physical model and numerical techniques I used; the Results section presents the findings from each experiment; and the Discussion interprets these results in the context of numerical accuracy and energy conservation.

METHODS

Background Science

To model the motion of each body in the Solar System, I assumed them as point-masses each inflicting gravitational forces on each other. I did this using the N-body force formula(4) below.

$$\vec{F}_i(t) = G \sum_{j=1, j\neq i}^{N} \frac{m_i m_j}{\|\vec{r}_{ji}(t)\|^2} \hat{r}_{ji}(t)$$

This equation shows the force of gravity exerted on body i by every other body $j \in \{1 \cdots N\}$ where $j \neq i$. I simplified this equation and arrived at the equation of motion for body i.

$$\vec{F}_i(t) = m_i \vec{a}_i(t) = G \sum_{j=1, j\neq i}^{N} \frac{m_i m_j}{\|\vec{r}_{ji}(t)\|^2} \hat{r}_{ji}(t)$$

$$\cancel{m_i} \vec{a}_i(t) = G \sum_{j=1, j\neq i}^{N} \frac{\cancel{m_i} m_j}{\|\vec{r}_{ji}(t)\|^2} \hat{r}_{ji}(t)$$

$$\vec{a}_i(t) = G \sum_{j=1, j\neq i}^{N} \frac{m_j}{\|\vec{r}_{ji}(t)\|^2} \left(\frac{\vec{r}_{ji}(t)}{\|\vec{r}_{ji}(t)\|} \right)$$

$$\vec{a}_i(t) = G \sum_{j=1, j\neq i}^{N} \frac{m_j}{\|\vec{r}_{ji}(t)\|^3} \vec{r}_{ji}(t)$$

$$\boxed{\vec{a}_i(t) = -G \sum_{j=1, j\neq i}^{N} \frac{m_j}{\|\vec{r}_{ij}(t)\|^3} \vec{r}_{ij}(t)}$$

Where $\vec{r}_{ij}(t) = \vec{r}_i(t) - \vec{r}_j(t)$ is the displacement from body i to body j. Now that I had the equation of motion, I had to find the value of G. For this simulation, I used: astronomical units(AU) for distances, years(yr) for time and Earth mass($M_\oplus$) for the masses of the bodies.

$$F = G \frac{m_i m_j}{r^2}$$

$$\cancel{m_i} \frac{4\pi^2 r}{T^2} = G \frac{\cancel{m_i} m_j}{r^2}$$

$$G = \frac{4\pi^2 r^3}{T^2 m_i}$$

Since the Earth takes a period of 1yr to rotate around the Sun at a radius of 1AU, we can substitute these values into the equation above and write out the Sun's mass in terms of Earth mass ($1M_\odot = 332946 M_\oplus$).

$$G = \frac{4\pi^2}{332946} [\mathrm{AU}^3 \mathrm{yr}^{-2} M_\oplus^{-1}]$$

A major assumption that I made is that the Sun is fixed at the origin throughout the simulation. In reality, the Sun orbits its barycenter(5) which makes it wobble. I chose to ignore this effect because it makes negligible difference to the Sun's gravitational forces.

Simulation Design

I implemented the simulation using an OOP design while making use of libraries like Matplotlib and NumPy. I chose to use Matplotlib(6) because of its ability to customise graphs and visualise data. I used NumPy(7) for its ability to use matrices which is integral for this simulation because of vectors like acceleration, velocity and position. The simulation contains two classes: **`Body`** and **`NBodySimulation`**.

Body stores the physical attributes of each body: name, mass, position, velocity, acceleration. Figure 1 shows a birds-eye view of the Solar System which is how I decided to make my simulation.

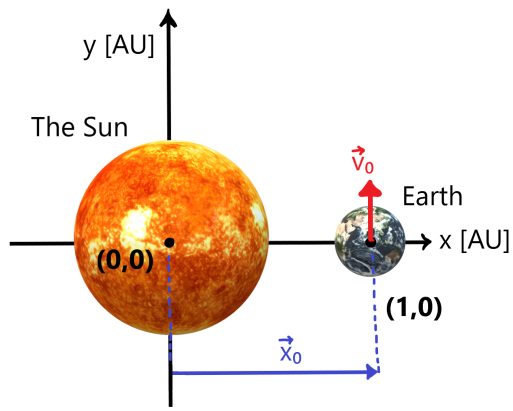

Figure 1. Birds-eye view of the Solar System.
The model that I implemented in the simulation.

This way, there are only 2 spatial dimensions which simplifies the problem. Figure 1 shows that for any body orbiting the Sun, the initial velocity and position would be.

$$\vec{v}_0 = \begin{bmatrix} 0 \\ \sqrt{\frac{GM_\odot}{R}} \end{bmatrix} \qquad \vec{x}_0 = \begin{bmatrix} R \\ 0 \end{bmatrix}$$

And for the Sun,

$$\vec{v}_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \qquad \vec{x}_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

I extracted details like orbital radii and masses of the planets from a file called `parameters_solar.json` using the **`load_bodies`** function which I made in the **`NBodySimulation`** class. **`NBodySimulation`** manages the simulation loop. It loads the initial conditions from a JSON file, uses various integration methods to calculate the planets' positions at each timestep, detects completed orbits, and computes the total energy of the system for 25 simulated years.

Integration Methods

I used three numerical integration methods for the simulation. The main difference between them is the choice of acceleration from a specific timestep.

Direct Euler(8) updates position using the current velocity($\vec{v}(t)$), and velocity using the current acceleration($\vec{a}(t)$).

$$\vec{r}(t+\delta t) = \vec{r}(t) + \vec{v}(t) \cdot \delta t$$
$$\vec{v}(t+\delta t) = \vec{v}(t) + \vec{a}(t) \cdot \delta t$$

Euler-Cromer(9) is a modification to Euler's method. It updates the current velocity $\vec{v}(t) \rightarrow \vec{v}(t+\delta t)$ and then uses this updated velocity to update position.

$$\vec{v}(t+\delta t) = \vec{v}(t) + \vec{a}(t) \cdot \delta t$$
$$\vec{r}(t+\delta t) = \vec{r}(t) + \vec{v}(t+\delta t) \cdot \delta t$$

This change makes Euler-Cromer good at energy conservation because it prevents the energy from drifting away from the equilibrium over long periods of time.

Beeman(10) uses both the current and previous accelerations to achieve higher accuracy when updating position and velocity.

$$\vec{r}(t+\delta t) = \vec{r}(t) + \vec{v}(t)\cdot\delta t + \frac{1}{6}\left[4\vec{a}(t) - \vec{a}(t-\delta t)\right]\cdot\delta t^2$$
$$\vec{v}(t+\delta t) = \vec{v}(t) + \frac{1}{6}\left[2\vec{a}(t+\delta t) + 5\vec{a}(t) - \vec{a}(t-\delta t)\right]\cdot\delta t$$

Beeman's method includes acceleration from the previous timestep, which is why I made attributes in the **`Body`** class for the current timestep's acceleration(**`self.acceleration`**) and the previous timestep's acceleration(**`self.prev_acceleration`**).

Experiment 1: Orbital Period Detection

I made a method inside the **`NBodySimulation`** class called **`detect_orbits`** which decides when a planet finishes its orbit to update said planet's **`last_crossing_time`** attribute. It does this every timestep throughout the whole simulation(**Figure 2**).

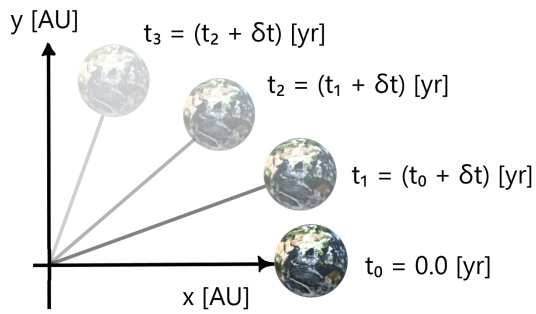

Figure 2. Diagram showing how the method tracks the position of planets each timestep.

The method considers every planet besides the Sun. At each timestep, it compares the current and previous y-coordinates of each planet. When the y-coordinate transitions from negative to positive while it's in motion, the planet has completed a full orbit. If this is the case, then the method stores the current timestep in that body's orbital period attribute. I simulated 25 years because Jupiter takes ≈ 11.86[yr] to complete 1 orbit(11). I wanted Jupiter to cross the x-axis at least twice for a reliable calculation.

Experiment 2: Energy Conservation

The total mechanical energy of the system at timestep t is $E(t) = T(t) + V(t)$. I implemented total energy using the formulae for T and V below.

$$T(t) = \sum_{i=1}^{N} \frac{1}{2} m_i \|\vec{v}_i(t)\|^2$$
$$V(t) = -G\sum_{i<j} \frac{m_i m_j}{\|\vec{r}_{ij}(t)\|}$$

To measure how well each integration method conserves energy, I calculated their RMS energy deviation

$$\delta E_{\text{RMS}} = \sqrt{\frac{1}{N}\sum_{k=1}^{N}(E_k - \bar{E})^2}$$

Where $\bar{E}$ is the mean total energy over all N timesteps. If $\delta E_{RMS} \approx 0$, this indicates better energy conservation, as it means the total energy stays closer to a constant value throughout the simulation.

Experiment 4: Planetary Alignment Detection

I made a method in the **`NBodySimulation`** class called **`detect_alignment`** which takes in a critical-angle ϕ as an input. It then computes the angles of each body against the x-axis and finds a mean angle (θ_{mean}).

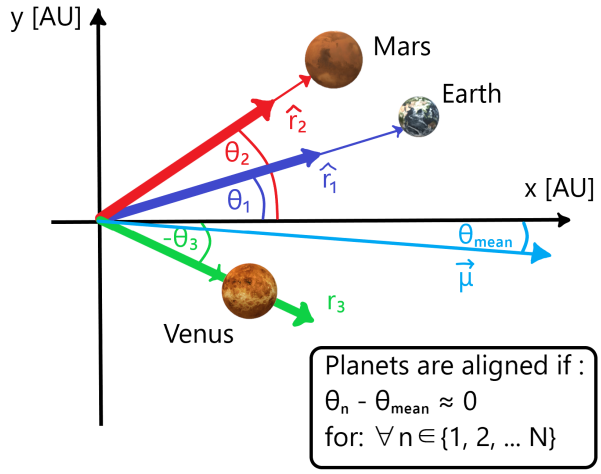

Figure 3. This diagram depicts how the method finds if the planets are close to alignment.

Then, the method checks if all the bodies make an angle $\delta\theta < \phi$ at each timestep using the equation below.

$$\delta\theta_i = |\text{arctan2}\,(\sin(\theta_i - \theta_{\text{mean}}), \cos(\theta_i - \theta_{\text{mean}}))| < \phi$$

I used unit vectors rather than angles to compute the mean to avoid wrapping errors near $\pm\pi$ which would cause issues when planets are near the $-x$-axis.

Assumptions and Limitations

I made a few assumptions to simplify this model. I assumed the planets to initially have circular orbits, I did not take relativistic effects into consideration. I treated the Sun as stationary body fixed at the origin, ignoring its motion due to gravitational interactions with the planets and its rotational property. Also I did not take into account gravitational effects from other celestial objects like comets, and moons.

RESULTS

Experiment 1: Orbital Period Accuracy

Table 1 shows the simulated orbital periods of each planet compared to NASA's calculations(12). In this experiment, I varied the timestep four times: $dt = 0.001, 0.0005, 0.00025, 0.000125$ (in [yr]) and ran the simulation using Beeman integration over 25 simulated years.

Table 1. Simulated orbital periods accuracy with *P.E* (Percentage Error)

Planet	$dt = 0.001$ **[yr]**	$dt = 0.0005$ **[yr]**	$dt = 0.00025$ **[yr]**	$dt = 0.000125$ **[yr]**
Mercury	0.241 (0.06%)	0.241 (0.06%)	0.241 (0.04%)	0.241 (0.04%)
Venus	0.614 (0.19%)	0.615 (0.03%)	0.615 (0.07%)	0.615 (0.07%)
Earth	1.000 (0.00%)	1.000 (0.00%)	1.000 (0.03%)	1.000 (0.01%)
Mars	1.881 (0.01%)	1.881 (0.01%)	1.881 (0.01%)	1.881 (0.01%)
Jupiter	11.823 (0.33%)	11.823 (0.33%)	11.823 (0.33%)	11.823 (0.33%)

All of the planets show percentage errors below 0.35% across all the timesteps tested. The simulation accuracy is highest for Earth and Mars with a $P.E \leq 0.03\%$. Jupiter has the largest error of $P.E = 0.33\%$ across every tested timestep.

Experiment 2: Energy Conservation

Figures 4 and 5 show my results from Experiment 2. Figure 4 shows the total energy($\Sigma E(t)$) vs time graphs for each of the integration methods throughout the whole simulation.

Beeman's method produces the smallest RMS energy deviation of $\delta E_{\text{RMS}} \approx 0.40 \times 10^{-6} [M_{\oplus} \text{AU}^2 \text{yr}^{-2}]$. Beeman's results seem to fluctuate between 2 bounds throughout the simulation. Direct Euler method shows energy continuously increasing throughout the entire simulation, with an RMS deviation of $\delta E_{\text{RMS}} \approx 1.04 \times 10^{7} [M_{\oplus} \text{AU}^2 \text{yr}^{-2}]$. This is roughly 1×10^{13} larger than Beeman's δE_{RMS}. Euler-Cromer shows more irregular and larger oscillations in total energy in comparison to Beeman. It has an RMS of $\delta E_{\text{RMS}} \approx 1228 \times 10^{-6} [M_{\oplus} \text{AU}^2 \text{yr}^{-2}]$.

Figure 5 shows the energy profile of the system as time passes (using Beeman's method). I varied the timestep three times $dt = 0.001, 0.0006, 0.0001$ (in [yr]). It is clear that as the timestep decreases, the energy fluctuations shrink down closer to a conserved quantity and the total energy appears constant. δE_{RMS} also shrinks to 0 as dt decreases.

Experiment 4: Planetary Alignment

Table 2 shows the results of experiment 4. The planetary alignment results. I varied both the critical-angle ϕ and the simulation length twice.

Table 2. Planetary alignment detection results for different threshold angles ϕ and simulation lengths.

ϕ (deg)	Simulation length [yr]	First alignment after $t = 0$ [yr]
5	25	None detected
5	500	None detected
10	25	None detected
10	250	$t \approx 145.250$

After running **`experiment_4.py`** the first few detections of alignments are guaranteed because all the planets start at the x-axis. According to Table 2, it is very rare to detect an alignment when $\phi = 5$ even after simulating 500 years. For $\phi = 10$, there was no alignment in the first 25 years. When simulating 250 years, I found an alignment after 145 years.

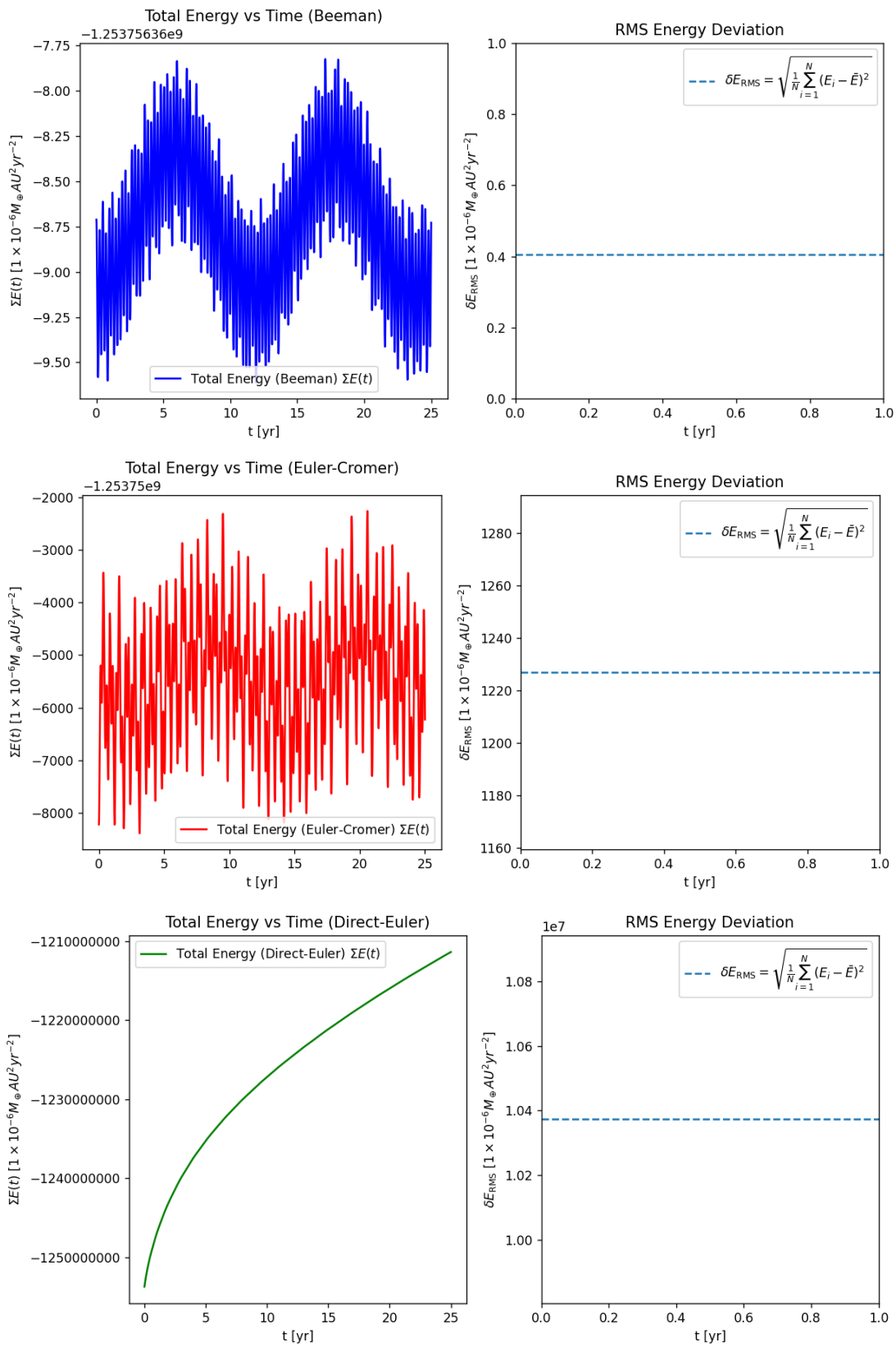

Figure 4. Total energy $\Sigma E(t)$ vs time t and RMS energy deviation δE_{RMS} graphs for Beeman, Euler-Cromer and Direct Euler where $dt = 0.001[\mathrm{yr}]$ over 25 years.

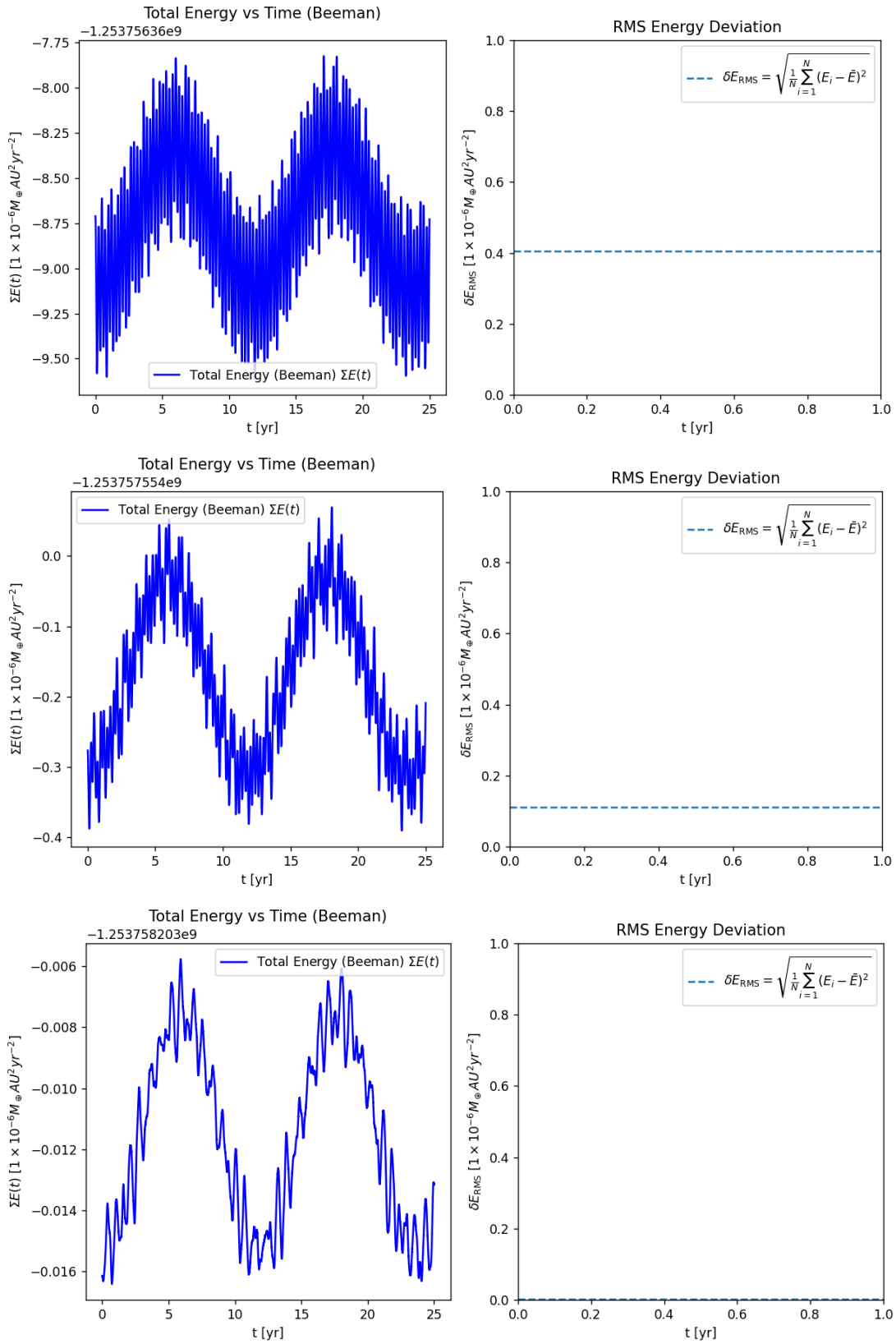

Figure 5. Total energy $\Sigma E(t)$ vs time and $\delta E_{\rm RMS}$ graphs for Beeman integration at $dt = 0.001$(top graph), 0.0006(middle graph) and 0.0001(bottom graph) in [yr]. Evidence of $\delta E_{\rm RMS}$ decreasing as dt shrinks.

DISCUSSION

Experiment 1: Orbital Period Accuracy

The simulated orbital periods are in agreement with NASA's calculated orbital periods for all the tested timesteps. I noticed Jupiter's orbital period consistently had a percentage error of $P.E = 0.33\%$. This may be because Jupiter only completes 2 orbits in the simulation but other planets complete many more, so their accuracies are higher. Another reason is because I assumed the Sun was fixed at the origin. In reality Jupiter's mass is large enough to cause the Sun to wobble around its barycenter, but my assumption makes that impossible here. Reducing dt from 0.001 to 0.000125 did not make much of a difference. The accuracies only improved by 0.02% on average. This likely means that the orbital period values converge at $dt = 0.001[\text{yr}]$. One effect that may have caused inaccuracies in orbital periods is when the planets get really close to eachother, the denominator term in Newton's universal law of gravitation becomes really close to 0 (because the distance between them is so small) which leads to a singularity. This can be avoided by using softening(14) as shown below.

$$\boxed{\vec{a}_i(t) = -G \sum_{j=1, j\neq i}^{N} \frac{m_j}{(\|\vec{r}_{ij}(t)\|^2 + \varepsilon^2)^{\frac{3}{2}}} \vec{r}_{ij}(t)}$$

Experiment 2: Energy Conservation

Beeman's method vastly outperforms both Euler-Cromer and Direct Euler at energy conservation. This is because Beeman's method is symplectic(13) which means, over long time-periods, the total energy of the system fluctuates around the system's true energy. Beeman's RMS deviation is roughly 1×10^3 times smaller than Euler's method. Out of all the methods, Euler's method had the least energy conservation because it kept over-estimating the energy needed for the next step.

Figure 5 shows the energy decreasing in orders of magnitude as I tested smaller values of dt. The RMS deviation also gradually approaches 0 as dt decreases. This is expected because as δE_{RMS} shrinks to 0, the energy fluctuates less around the equilibrium, therefore energy is conserved well.

Experiment 4: Planetary Alignment

The early alignment detections at $t = 0.001, 0.002\ldots$ yr are not relevant since all the planets started at the x-axis. A more efficient implementation of this method would ignore detections within the first few years of the simulation.

The first proper planetary alignment occurs at a critical-angle of $\phi = 10$ at $t \approx 145.250[\text{yr}]$. which is because $\phi = 5$ is such a small gap to have every body make an angle of less than ϕ against a common line. Extending the simulation further to 1000+ years or reducing dt would improve accuracy in these results.

CONCLUSION

I developed an N-body simulation of the Solar System using three numerical integration methods and used it to investigate orbital period accuracy, energy conservation, and planetary alignment. From conducting experiments on my simulation, my main findings are shown below.

- Beeman integration reproduces planetary orbital periods to within 0.35% of NASA's accepted values across all timesteps tested, with the simulation largely converged at $dt = 0.001$ yr for the inner planets.
- Jupiter's orbital period error of 0.33% is likely an effect of fixing the Sun at the origin instead of mimicking how it actually behaves.
- Beeman's method conserves energy far better than both Direct Euler and Euler-Cromer at very small steps of dt (in [yr]). With an RMS deviation roughly three orders of magnitude smaller than Direct Euler.

- Reducing the timestep in Beeman's method consistently improves energy conservation, with the RMS deviation approaching zero at $dt = 0.0001$ yr.
- A planetary alignment within $\phi = 10$ was detected at $t \approx 145$ yr, but no alignment was found for $\phi = 5$ within 100 years.

ACKNOWLEDGEMENTS

I would like to thank my course instructor Britton Smith as well as the members of the course team for their consistent help every week in helping me understand OOP design and step-wise simulation in python.

Word count: 20045

REFERENCES

[1] Bishop, R. (2015). Chaos (Stanford Encyclopedia of Philosophy). [online] Stanford.edu. Available at: https://plato.stanford.edu/entries/chaos/.

[2] PYTHON (2025). Python. [online] Python.org. Available at: https://www.python.org/.

[3] NASA (2018). Planet Compare – NASA Solar System Exploration. [online] NASA Solar System Exploration. Available at: https://solarsystem.nasa.gov/planet-compare/.

[4] Heggie, D. (n.d.). Computation and astrophysics of the N-body problem. [online] Available at: http://webhomes.maths.ed.ac.uk/~heggie/lectures1+2.pdf [Accessed 6 Apr. 2026].

[5] NASA (2016). What is a barycenter? — NASA Space Place – NASA Science for Kids. [online] Nasa.gov. Available at: https://spaceplace.nasa.gov/barycenter/en/.

[6] Matplotlib (2024). Matplotlib: Python Plotting — Matplotlib 3.1.1 Documentation. [online] Matplotlib.org. Available at: https://matplotlib.org/.

[7] Numpy (2024). NumPy. [online] Numpy.org. Available at: https://numpy.org/.

[8] Dawkins, P. (2018). Differential Equations - Euler's Method. [online] tutorial.math.lamar.edu. Available at: https://tutorial.math.lamar.edu/Classes/DE/EulersMethod.aspx.

[9] Physlets.org. (2026). Modified Euler algorithms. [online] Available at: https://sip.physlets.org/SimpleEPubse13.html [Accessed 6 Apr. 2026].

[10] Google.com. (2026). Available at: https://www.google.com/url?sa=t&source=web&rct=j&opi=89978449&url=https://nctech-lab.ru/Content/ArticleImages/cd0897f8-3be5-4b63-9bcc-88dfce19ba26.pdf&ved=2ahUKEwiwlYqDi9qTAxUyUkEAHUmJB88QFnoECC4QAQ&usg=AOvVaw1IWMK9n8cyDw31puZBMZSX [Accessed 6 Apr. 2026].

[11] NASA (2025). Jupiter: Facts - NASA Science. [online] science.nasa.gov. Available at: https://science.nasa.gov/jupiter/jupiter-facts/.

[12] NASA (2018). Planet Compare – NASA Solar System Exploration. [online] NASA Solar System Exploration. Available at: https://solarsystem.nasa.gov/planet-compare/.

[13] Wikipedia Contributors (2025). Symplectic integrator. Wikipedia.

[14] Heggie, D. (n.d.). Computation and astrophysics of the N-body problem. [online] Available at: https://webhomes.maths.ed.ac.uk/~heggie/lectures1+2.pdf [Accessed 7 Apr. 2026].